

TECHNISCHE UNIVERSITÄT
CHEMNITZ

AI-Augmented FAIRness Assessment of Research Software: A Hybrid Approach

Master Thesis

Submitted in Fulfilment of the
Requirements for the Academic Degree
M.Sc.

Dept. of Computer Science
Chair of Computer Engineering

Submitted by: Klea Haxhiu
Student ID: [your student ID]
Date: August 23, 2026

Supervising tutor: [your supervisor name]

Abstract

Keywords: FAIR, research software, LLM, documentation quality, FAIR-ness assessment

Contents

Contents **4**

List of Figures **7**

List of Tables **8**

List of Abbreviations **9**

1 Introduction **10**
1.1 Motivation 10
1.2 Problem Statement 11
1.3 Research Objective and Research Questions 11
1.4 Thesis Structure 12

2 State of the Art **13**
2.1 The FAIR Principles 13
2.2 FAIR for Research Software 14
2.3 Existing FAIRness Assessment Approaches 15
2.3.1 F-UJI 15
2.3.2 FAIR-IMPACT Metrics 15
2.3.3 FAIR-Aware 16
2.3.4 FAIRsharing 16
2.3.5 Summary 16
2.4 Software Documentation Quality 17
2.4.1 README Content and Structure 17
2.4.2 Beyond Accuracy: Multi-Dimensional Documentation Quality 17
2.4.3 Practitioner Perspective on Documentation Quality 17
2.4.4 Summary 18
2.5 LLM-Based Document Analysis 18
2.5.1 LLM-Based FAIR Assessment 18
2.5.2 LLMs for Software Engineering 19
2.5.3 LLM-Based Code Documentation Assessment 19
2.5.4 LLM-as-Judge 20
2.5.5 Summary 20
2.6 Research Gap 20

3 Methodology and System Design **22**
3.1 Overall Research Design 22
3.2 Dataset Collection 22
3.2.1 Research Software Definition 22
3.2.2 Collection Strategy 23
3.2.3 Filtering and Cleaning 23
3.2.4 Dataset Characteristics 23
3.3 Ground Truth Construction 24
3.3.1 Purpose and Scope 24
3.3.2 Repository Selection 24
3.3.3 Annotation Guidelines 24
3.3.4 Annotation Process 25
3.3.5 Inter-Annotator Agreement 25
3.4 Rule-Based Assessment 25
3.4.1 Overview 25
3.4.2 Indicators 26
3.4.3 Implementation 26
3.4.4 Output 27
3.5 LLM-Based Assessment 27
3.5.1 Overview 27
3.5.2 Documentation Quality Dimensions 27
3.5.3 Model Selection 28
3.5.4 Prompt Design 28
3.5.5 Implementation 29
3.5.6 External Link Policy 29
3.5.7 Output 29
3.6 Repository FAIRness Profile 30
3.7 Evaluation Strategy 30
3.7.1 Ground Truth Comparison 30
3.7.2 Inter-Annotator Agreement 31
3.7.3 Baseline Comparison 31
3.7.4 Qualitative Error Analysis 31
3.7.5 Confidence Calibration 31
3.7.6 README Length and Verbosity Bias 31

4 Implementation **33**

5 Evaluation and Results **34**
5.1 Experimental Setup 34
5.1.1 Dataset 34
5.1.2 Evaluation Metrics 34
5.1.3 Models and Tools 35
5.1.4 Experimental Environment 35
5.1.5 Evaluation Procedure 35

5.2 Ground Truth Validation . . . 36
5.2.1 Annotation Process . . . 36
5.2.2 Inter-Annotator Agreement . . . 36
5.3 Rule-Based Assessment Results . . . 38
5.3.1 Full Dataset Results . . . 38
5.3.2 Validation Against Ground Truth . . . 39
5.4 LLM-Based Assessment Results . . . 40
5.4.1 LLM vs Primary Annotator . . . 40
5.4.2 LLM vs Second Annotator . . . 40
5.4.3 Systematic Scoring Patterns . . . 40
5.4.4 Comparison with Inter-Annotator Agreement . . . 41
5.4.5 Confidence Calibration . . . 42
5.4.6 README Length and Verbosity Bias . . . 43
5.5 Hybrid FAIRness Profile Analysis . . . 43
5.5.1 Score Distributions . . . 44
5.5.2 Relationship Between Rule-Based and LLM Scores . . . 44
5.5.3 Complementary Assessment Dimensions . . . 44
5.5.4 Profile Interpretation . . . 45
5.6 Baseline Comparison . . . 45
5.6.1 F-UJI Baseline Applicability . . . 45
5.6.2 howfairis Assessment Results . . . 46
5.6.3 Scope Comparison . . . 47
5.7 Summary of Findings . . . 47
5.7.1 Ground Truth Reliability . . . 48
5.7.2 Rule-Based Assessment . . . 48
5.7.3 LLM-Based Assessment . . . 48
5.7.4 Hybrid FAIRness Profile . . . 48
5.7.5 Baseline Comparison . . . 48
5.7.6 Research Question Responses . . . 49

6 Discussion . . . 51

7 Conclusion . . . 52

Bibliography . . . 53

List of Figures

List of Tables

5.1 Inter-annotator agreement for rule-based indicators and AI-based dimensions (n=30). 37
5.2 Rule-based indicator frequencies across the full dataset (n=188). . . . 38
5.3 Distribution of rule-based total scores across the full dataset (n=188). 38
5.4 Rule-based validation against manual ground truth (n=30). 39
5.5 LLM vs primary annotator agreement (n=30). 40
5.6 LLM vs second annotator agreement (n=30). 40
5.7 Direction of LLM scoring disagreements relative to primary annotator (n=30). LLM lower = LLM score < human score; LLM higher = LLM score > human score. 41
5.8 Agreement summary: human-human vs LLM-human (macro-avg weighted κ, AI dimensions only). 41
5.9 LLM confidence calibration against the primary human annotations across all documentation dimensions. 42
5.10 Mean scores for rule-based and LLM-based components across the 30 ground truth repositories. 44
5.11 F-UJI assessment results compared with the proposed framework's rule-based and LLM documentation scores (n=5). 46
5.12 howfairis score distribution across the 30 ground truth repositories. . 46
5.13 Comparison of assessment scope across howfairis, the rule-based component, and the LLM-based component. 47

List of Abbreviations

FAIR Findable, Accessible, Interoperable, Reusable

FAIR4RS FAIR for Research Software

LLM Large Language Model

API Application Programming Interface

README Read Me

DOI Digital Object Identifier

1 Introduction

1.1 Motivation

The increasing importance of open science and reproducibility has led to a growing need for effective management and reuse of digital research outputs, including data, software, and computational workflows. The FAIR principles — Findable, Accessible, Interoperable, and Reusable — were introduced by Wilkinson et al. [11] to address these challenges by providing guidelines that enhance the discoverability and reuse of research artifacts. A central goal of the FAIR principles is machine-actionability — the ability of automated agents to discover, evaluate, and act on digital objects without prior knowledge of them.

While initially developed for research data, the FAIR principles are equally applicable to research software, which plays a central role in modern scientific processes. The FAIR for Research Software (FAIR4RS) principles [1] extend and adapt the original framework to address the specific characteristics of software, such as executability, versioning, and dependency management. Ensuring that software is FAIR is essential for enabling reproducibility, transparency, and collaboration.

However, in practice, many research software repositories lack essential elements including structured metadata, clear licensing information, persistent identifiers, and comprehensive documentation. Assessing the FAIRness of research software at scale remains a challenging task, particularly for indicators that depend on the quality and completeness of software documentation [2].

Existing FAIRness assessment approaches primarily rely on rule-based checks that evaluate the presence of metadata files, licenses, persistent identifiers, and other objectively measurable indicators [4]. While these approaches provide valuable automated assessments, they often struggle to evaluate qualitative aspects such as installation guidance, usage instructions, and documentation quality — factors that directly influence software reuse and reproducibility [2].

Recent advances in Large Language Models (LLMs) offer new opportunities for analyzing natural language documentation and supporting FAIRness assessment beyond traditional rule-based methods. By interpreting README files and other documentation artifacts, LLMs have the potential to evaluate aspects that are difficult to capture using deterministic rules alone.

This thesis investigates how AI-based documentation assessment can complement rule-based FAIRness evaluation for research software through a hybrid assessment approach that combines automated file-based checks with LLM-supported analysis of repository documentation. The goal is to explore whether AI-assisted assessment

can improve the evaluation of qualitative FAIR-related aspects that are difficult to capture using existing rule-based methods alone.

1.2 Problem Statement

Current methods for evaluating the FAIRness of research software repositories are limited in their ability to scale and to assess unstructured information. Rule-based approaches, such as those implemented in F-UJI [4], can effectively detect explicit features such as the presence of a license file or structured metadata, but are insufficient for evaluating qualitative aspects such as documentation clarity, completeness, and usability. As noted by Chue Hong et al. [2], automated tools can verify the existence of metadata but cannot reliably determine its quality, relevance, or usefulness.

Research on Mining Software Repositories has shown that README files are the primary documentation artifact for GitHub repositories and contain rich information about a project's purpose, usage, and quality [8]. However, the systematic integration of documentation-based information into automated FAIRness assessment workflows remains limited. Rule-based tools can verify whether a README file exists, but cannot determine whether it adequately describes installation procedures, usage examples, or software dependencies. Treude et al. [10] further demonstrate that documentation quality goes beyond mere presence — dimensions such as clarity, understandability, and structure require qualitative assessment that rule-based approaches cannot provide.

The importance of documentation quality extends beyond academic frameworks. Imani et al. [6] found that over 91% of industry practitioners rely on OSS documentation for software adoption decisions, and that the top reported challenge is not the absence of documentation but the difficulty of finding relevant information within it. This practitioner evidence reinforces the need for assessment methods that evaluate not just the presence but the quality and completeness of documentation content.

Furthermore, the lack of automated and scalable solutions makes it difficult to assess large numbers of repositories consistently. Consequently, there is a need to investigate whether a hybrid approach that combines rule-based structural checks with AI-based qualitative assessment can provide a more comprehensive evaluation of research software FAIRness while remaining scalable and automated.

1.3 Research Objective and Research Questions

The primary objective of this thesis is to design and implement a prototype system for the automated assessment of FAIRness in research software repositories hosted on GitHub. The system combines rule-based indicators derived from the FAIR4RS principles [1] with an AI-based documentation assessment component, and evaluates whether this hybrid approach provides a more comprehensive FAIRness evaluation

than rule-based methods alone.

The following research questions guide the investigation:

RQ1 To what extent can LLM-based assessment of README documentation improve FAIRness scoring compared to rule-based methods alone?

RQ2 What are the reliability and failure modes of LLM-based FAIRness assessment, and what types of repositories or documentation styles cause errors or produce unreliable results?

The system is implemented as a modular Python prototype focused on GitHub-hosted repositories. Extensibility to other platforms such as Zenodo or Codeberg is considered out of scope for this thesis but is noted as a direction for future work.

1.4 Thesis Structure

The remainder of this thesis is organized as follows.

Chapter 2 reviews the relevant literature, covering the FAIR principles and their adaptation for research software, existing automated FAIRness assessment tools, research on software documentation quality, and recent advances in LLM-based document analysis.

Chapter 3 describes the proposed hybrid assessment approach, including the dataset collection methodology, the rule-based component, the AI-based documentation assessment component, and the scoring and output design.

Chapter 4 presents the evaluation strategy and results, including the ground truth construction process, quantitative evaluation against manual annotations, and qualitative error analysis of the AI-based component.

Chapter 5 discusses the findings in relation to the research questions, reflects on the limitations of the approach, and outlines directions for future work.

Chapter 6 concludes the thesis with a summary of contributions and key findings.

2 State of the Art

2.1 The FAIR Principles

The FAIR principles were introduced by Wilkinson et al. [11] as a minimal, domain-independent framework for improving the findability, accessibility, interoperability, and reusability of digital research objects. The principles were designed to support both human and machine interaction with research outputs, with machine-actionability as a central goal — the ability of automated agents to discover, evaluate, and act on digital objects without prior knowledge of them.

The four FAIR principles are defined as follows.

A digital object is **Findable** if it is assigned a globally unique and persistent identifier, described with rich metadata, and indexed in a searchable resource.

It is **Accessible** if it can be retrieved via an open and standardized protocol, and if its metadata remains accessible even when the object itself is no longer available.

Interoperability requires that the object uses formal knowledge representation languages, FAIR vocabularies, and qualified references to other objects.

Finally, an object is **Reusable** if it has rich attributes, a clear license, detailed provenance, and meets community standards.

An important characteristic of the FAIR principles is that they are modular and independent — they can be adopted incrementally and in any combination. Furthermore, FAIRness is treated as a continuum rather than a binary property: a digital object can be more or less FAIR depending on how completely it satisfies each principle. This continuum directly motivates the development of automated assessment tools capable of measuring FAIRness at scale.

The FAIR principles have become one of the most influential frameworks in research data management and open science. They have been adopted by funding agencies, research infrastructures, publishers, and scientific communities worldwide as a reference framework for improving the management and reuse of digital research outputs. Their widespread adoption has also motivated the development of FAIR assessment tools, metrics, and maturity indicators.

Despite their influence, the FAIR principles intentionally do not prescribe specific technologies, implementation strategies, or minimum compliance requirements. As a result, determining the degree of FAIRness of a digital object can be challenging and often requires interpretation. This has led to the development of automated assessment frameworks and community-specific implementations that translate the principles into measurable indicators.

While the FAIR principles were originally developed for research data, Wilkinson

et al. explicitly noted their applicability to other scholarly digital objects including software, workflows, and algorithms. Consequently, while the FAIR principles provide a general conceptual framework, additional guidance is required to address software-specific characteristics and to enable meaningful FAIRness assessment of research software repositories.

2.2 FAIR for Research Software

Although the FAIR principles were originally formulated for research data, software has become an equally important research output in modern science. Research software differs from data in several ways, including its executable nature, dependency structure, versioning requirements, and ongoing maintenance needs. As a result, direct application of the original FAIR principles to software can be challenging.

To address these challenges, the FAIR for Research Software (FAIR4RS) principles were introduced by Barker et al. [1]. The FAIR4RS principles were developed through a large community consultation process involving approximately 500 contributors from more than 110 organisations, reflecting broad international consensus on the need for software-specific FAIR guidance. The principles adapt the original framework to account for the unique characteristics of software, introducing new sub-principles where necessary.

Under FAIR4RS, software is **Findable** if it is assigned globally unique and persistent identifiers, with distinct identifiers for different versions and components — an extension not present in the original FAIR principles. Software is **Accessible** if it can be retrieved via standardised protocols and its metadata remains available even if the software itself becomes unavailable. **Interoperability** in FAIR4RS extends beyond metadata to include API interactions and references to related software objects. Finally, software is **Reusable** if it is executable, understandable, modifiable, and integrable into other systems, with clear licensing, provenance, and community standards.

Research software plays a critical role in enabling reproducible science, and FAIR4RS therefore positions software discoverability, accessibility, and reusability as essential requirements for transparent and reproducible research workflows.

A key insight of FAIR4RS is that FAIRness for software should begin during development rather than only at publication time. Unlike data, software may be used directly from its development environment, making early adoption of FAIR practices essential for reproducibility and reuse. The principles have received growing institutional support from organisations such as ELIXIR, ARDC, the Netherlands eScience Center, and ZB MED.

Despite this progress, FAIR4RS defines high-level principles without prescribing concrete automated assessment methods. Many FAIR4RS indicators rely on metadata and structural information, while qualitative aspects such as documentation clarity, installation guidance, and usage examples remain difficult to evaluate automatically. This gap between the principles and their practical assessment motivates

the development of automated tools capable of evaluating both structured and unstructured aspects of research software repositories.

2.3 Existing FAIRness Assessment Approaches

Several tools and frameworks have been developed to automate the assessment of FAIRness for digital research objects. This section reviews the most relevant approaches, highlighting their capabilities and limitations in the context of research software assessment.

2.3.1 F-UJI

F-UJI is an automated web-based FAIR assessment tool developed within the FAIRsFAIR initiative [4]. It evaluates the FAIRness of digital research objects by performing machine-actionable checks against predefined metrics, producing quantitative scores across the four FAIR dimensions. F-UJI accepts persistent identifiers and metadata endpoints as input and evaluates indicators such as metadata presence, persistent identifiers, licensing, semantic vocabularies, provenance information, and accessibility protocols.

F-UJI treats FAIRness as a spectrum rather than a binary property, providing detailed metric-level reporting and evidence traces. It demonstrates that many aspects of FAIR compliance can be assessed automatically through metadata checks. However, F-UJI is primarily metadata-centric and was originally designed for FAIR digital objects and datasets rather than research software repositories specifically. It was not designed for direct GitHub repository assessment and expects persistent identifiers and structured metadata records as input, which many repositories in practice do not provide.

Critically, F-UJI evaluates the existence and machine-readability of metadata rather than its semantic quality or usefulness. Documentation quality, understandability, and usability are not assessed, creating a gap between structural FAIR compliance and meaningful software reusability.

2.3.2 FAIR-IMPACT Metrics

The FAIR-IMPACT initiative extended existing FAIR assessment approaches by defining 17 concrete, measurable FAIR Research Software Metrics (FRSM-01 to FRSM-17) for automated assessment of research software against the FAIR4RS principles [2]. These metrics operationalize FAIR4RS into practical automated checks, covering findability, accessibility, interoperability, and reusability indicators at the code, project, and repository level.

The metrics use three compliance levels — Essential, Important, and Useful — and are designed to be domain-agnostic while adaptable to disciplinary contexts. Planned integration into F-UJI demonstrates their practical application potential.

However, the FAIR-IMPACT metrics explicitly acknowledge that automated tools can verify the existence of metadata but cannot reliably determine its quality, relevance, or correctness. Qualitative aspects such as documentation clarity and usability remain outside the scope of automated rule-based assessment.

2.3.3 FAIR-Aware

In contrast to automated assessment tools, FAIR-Aware is a human-centred self-assessment tool developed within the FAIRsFAIR initiative that focuses on increasing FAIR literacy among researchers. It uses a questionnaire-based approach to help researchers reflect on how FAIR practices can be integrated into their workflows, providing guidance and recommendations rather than automated compliance metrics.

FAIR-Aware demonstrates that FAIRness assessment is not limited to verifying machine-readable metadata. Many FAIR considerations require researchers to interpret community standards, evaluate reuse conditions, and make contextual decisions that depend on domain knowledge and qualitative judgment. This highlights the importance of qualitative reasoning in FAIR evaluation — a dimension that automated rule-based tools cannot fully address.

2.3.4 FAIRsharing

FAIRsharing is a curated registry of FAIR-related standards, repositories, databases, policies, and metadata schemas. It provides centralised infrastructure to support the discovery, adoption, and interconnection of community standards and FAIR-enabling resources across research domains. FAIRsharing demonstrates that FAIRness is deeply connected to the existence and adoption of shared standards and controlled vocabularies — automated FAIR assessment systems rely heavily on this ecosystem infrastructure to validate interoperability and machine-readability. FAIRsharing is particularly relevant for research software because it supports the discovery of community standards, metadata schemas, and domain-specific resources that contribute to software interoperability and reuse.

2.3.5 Summary

Existing FAIRness assessment tools provide valuable automated evaluation capabilities and have significantly advanced the practical application of FAIR principles. However, their primary focus remains on structured metadata and objectively measurable indicators, leaving qualitative aspects of software documentation largely unexplored.

2.4 Software Documentation Quality

While existing FAIRness assessment tools focus on structured metadata and file presence, research on software documentation quality highlights a complementary dimension of software reusability that these tools do not address. README files, in particular, serve as the primary documentation artifact for GitHub repositories and play a central role in how users discover, understand, and reuse software.

2.4.1 README Content and Structure

Prana et al. [8] conducted the first large-scale empirical study of GitHub README files, categorising their content into eight types: What, Why, How, When, Who, References, Contribution, and Other. Their study of 393 repositories found that while installation and usage instructions (How) are present in the majority of files, purpose and motivation sections (Why) appear in only 25.7% of repositories, and project status information (When) in only 21.4%.

This systematic absence of purpose and status information is not merely a documentation style issue — it reflects a gap in communicating software maturity and intent that directly maps to FAIR4RS reusability requirements. Rule-based tools can detect whether a README file exists, but cannot determine whether it contains the information a user needs to evaluate or reuse the software.

2.4.2 Beyond Accuracy: Multi-Dimensional Documentation Quality

Treude et al. [10] proposed a ten-dimensional framework for assessing software documentation quality, adapted from the information quality literature. The framework deliberately goes beyond accuracy and completeness — the dominant focus of prior work — to cover dimensions such as clarity, understandability, structure, readability, and conciseness.

A key theoretical contribution of this framework is the deliberate exclusion of accuracy and completeness as primary dimensions, since these require external reference points. This means that even a complete README — one that passes rule-based checks — can still score poorly on clarity or understandability. The framework directly supports the argument that rule-based FAIRness assessment is insufficient on its own, and that dimensions such as clarity and understandability require a qualitative assessment approach.

2.4.3 Practitioner Perspective on Documentation Quality

Imani et al. [6] provided empirical evidence from an industry perspective, finding that over 91% of practitioners rely on OSS documentation for software adoption decisions. Their study identified that the top reported challenge is not the absence of documentation but the difficulty of finding relevant information within it. Nine

adoption criteria emerged from practitioner interviews, including ease of use, compatibility, functionality, and presence of usage examples — criteria that closely map to FAIR4RS reusability indicators.

This practitioner evidence reinforces the need for assessment methods that evaluate not just the presence but the quality and completeness of documentation content.These findings suggest that structured rubrics and explicit scoring criteria are important design considerations for LLM-based assessment systems, particularly when consistency and reliability are required.

2.4.4 Summary

Research on software documentation quality consistently demonstrates that documentation presence alone is insufficient to ensure software reusability. The content, clarity, and completeness of documentation are critical factors that current automated FAIRness assessment tools do not evaluate. This creates a clear motivation for approaches that combine traditional rule-based indicators with qualitative assessment of documentation content.

2.5 LLM-Based Document Analysis

Recent advances in large language models (LLMs) have opened new possibilities for automated analysis of natural language artifacts in software repositories. This section reviews work on LLM capabilities for software engineering tasks, code documentation generation, and automated evaluation — providing the technical foundation for the AI-based component of this thesis.

2.5.1 LLM-Based FAIR Assessment

A closely related contribution is Fair-Way [9], an open-source framework that applies Large Language Models (LLMs) to automate FAIRness assessment of research datasets. Unlike traditional rule-based tools, Fair-Way uses LLMs to evaluate FAIRsFAIR Data Object Assessment Metrics (DOAM), aiming to improve flexibility across heterogeneous metadata formats and domains.

Fair-Way follows a divide-and-conquer strategy, decomposing the assessment into fine-grained tasks while splitting large metadata files into manageable chunks. The system employs prompt engineering techniques including structured JSON output, task decomposition, chain-of-thought prompting, and few-shot examples to improve assessment reliability and reproducibility. The authors evaluate multiple LLMs, reporting GPT-4o as the best-performing model, and compare Fair-Way against F-UJI on three datasets from different scientific domains. Their results demonstrate performance comparable to, and in several assessment metrics exceeding, the rule-based baseline.

Despite these promising results, Fair-Way differs substantially from the approach proposed in this thesis. First, it focuses on research datasets and FAIRsFAIR DOAM metrics, whereas this work targets research software repositories and FAIR4RS-inspired documentation indicators. Second, Fair-Way relies on a purely LLM-based assessment pipeline, while the authors explicitly identify hybrid approaches combining LLMs with explicit rule-based verification as an important direction for future work. Finally, Fair-Way evaluates dataset metadata, whereas this thesis combines rule-based repository analysis with LLM-based assessment of GitHub repository documentation to produce a hybrid FAIRness evaluation for research software.

These differences highlight an important gap in the current literature. While Fair-Way demonstrates the potential of LLMs for automated FAIR assessment, no existing approach combines rule-based repository analysis with LLM-based documentation assessment for research software repositories. The proposed hybrid approach presented in this thesis is intended to address this gap.

2.5.2 LLMs for Software Engineering

Hou et al. [5] conducted a systematic literature review of LLM applications across the software engineering lifecycle, finding that LLMs demonstrate strong performance on tasks involving natural language artifacts such as documentation, code summarization, and explanation generation. A central argument of the review is that many software engineering tasks involving documentation, explanation, and comprehension are fundamentally language understanding problems that benefit from the semantic reasoning capabilities of LLMs.

The review also identifies a consistent recommendation across studies: hybrid approaches combining LLMs with traditional methods are preferable for reliability-critical applications, as LLM limitations such as hallucination, prompt sensitivity, and inconsistent outputs warrant complementary rule-based checks. This directly supports the hybrid architecture proposed in this thesis.

2.5.3 LLM-Based Code Documentation Assessment

Dvivedi et al. [3] conducted a comparative study of multiple LLMs for code documentation generation, evaluating outputs on six quality dimensions including accuracy, completeness, relevance, understandability, and readability. Their study found that closed-source models consistently outperform open-source alternatives, and that file-level documentation — analogous to README files — is consistently the most challenging documentation type for LLMs to assess reliably.

The study also validated that checklist-based evaluation rubrics reduce inter-rater subjectivity, achieving only 11.67% disagreement across raters. This finding supports the use of structured rubrics in LLM-based scoring — the approach adopted in this thesis for evaluating README documentation quality against FAIR4RS indicators.

2.5.4 LLM-as-Judge

Zheng et al. [12] conducted the first systematic empirical study of the LLM-as-a-judge paradigm, evaluating the use of strong LLMs as automated evaluators of open-ended responses. They found that GPT-4 as judge achieves over 80% agreement with human raters — matching human-human agreement levels — validating LLM-based scoring as a credible and scalable evaluation method for open-ended quality assessment tasks.

The study also characterised key biases of LLM judges, including position bias, verbosity bias, and self-enhancement bias. For single answer grading — the scenario most relevant to README quality assessment — the study found that absolute scores are less stable than relative comparisons, and that providing structured rubrics with explicit criteria substantially improves scoring reliability. These findings directly inform the prompt design and confidence flagging mechanism developed in this thesis.

2.5.5 Summary

LLMs offer strong capabilities for analyzing natural language documentation and assessing qualitative aspects of software artifacts that rule-based approaches cannot evaluate. However, existing work also highlights consistent limitations including output instability, prompt sensitivity, and domain-specific challenges. These limitations motivate the need for uncertainty quantification and confidence flagging in any LLM-based assessment system.

2.6 Research Gap

The literature reviewed in this chapter reveals a clear and consistent gap in existing approaches to FAIRness assessment of research software.

Automated FAIRness assessment tools such as F-UJI and the FAIR-IMPACT metrics have made significant progress in evaluating structured, machine-readable aspects of research software repositories, including metadata presence, persistent identifiers, licensing, and dependency specifications. However, these tools predominantly rely on rule-based checks that detect the presence or absence of explicit repository features. They do not analyse the quality or content of unstructured documentation such as README files, which research has shown to be critical indicators of software reusability and usability [8, 10].

Research on software documentation quality demonstrates that dimensions such as clarity, understandability, and completeness of installation and usage instructions are important for software adoption and reuse [6], yet these dimensions are outside the scope of existing automated FAIRness assessment approaches. The FAIR-IMPACT metrics explicitly acknowledge that automated tools can verify the existence of metadata but cannot reliably determine its quality or usefulness [2].

Recent advances in large language models offer a promising approach to filling this gap. LLMs have demonstrated strong capabilities for analyzing natural language documentation and evaluating qualitative aspects of software artifacts [5, 3]. The LLM-as-a-judge paradigm has been validated as a scalable alternative to human evaluation for open-ended quality assessment tasks [12]. Despite this potential, the integration of LLMs into FAIRness assessment workflows remains largely unexplored. No existing approach systematically combines rule-based FAIR indicators with LLM-based documentation quality assessment for research software repositories.

This thesis addresses this gap by designing, implementing, and evaluating a hybrid assessment approach that combines automated rule-based checks with LLM-based analysis of README documentation against FAIR4RS reusability and findability indicators. The approach aims to complement existing FAIRness assessment tools by providing qualitative evaluation of documentation dimensions that rule-based methods cannot assess, while also investigating the reliability and limitations of LLM-based assessment.

3 Methodology and System Design

3.1 Overall Research Design

This thesis proposes and evaluates a hybrid approach to automated FAIRness assessment of research software repositories. The approach combines two complementary assessment components: a rule-based component that evaluates the presence of structured metadata and repository files against indicators derived from the FAIR4RS principles [1], and an AI-based component that evaluates the quality and completeness of README documentation using a Large Language Model. Together, these components provide complementary evidence that forms the basis for the repository FAIRness assessment.

The motivation for a hybrid design stems from the limitations of existing approaches. Rule-based tools such as F-UJI [4] and howfairis assess the presence or absence of structured metadata — license files, persistent identifiers, citation files, and dependency specifications — but cannot evaluate qualitative characteristics of documentation, such as whether installation instructions, usage guidance, examples, or contribution information are sufficiently complete for software reuse. The rule-based component produces deterministic and reproducible assessments, while the LLM-based component captures qualitative aspects of documentation that are difficult to express through fixed rules.

The research design follows four main phases. First, a dataset of 188 research software repositories is collected from GitHub using a systematic topic-based search strategy. Second, a ground truth set of 30 repositories is manually annotated against a structured rubric derived from FAIR4RS principles, providing a reference for evaluating the accuracy of both assessment components. Third, the rule-based and LLM-based components are implemented, validated against the ground truth annotations, and applied to the full dataset. Fourth, the outputs of both components are evaluated to assess agreement between automated assessments and manual annotations, and to compare the proposed approach with existing baseline tools.

The following sections describe each component of the methodology in detail.

3.2 Dataset Collection

3.2.1 Research Software Definition

Research software is defined in this work as software developed to support, enable, or conduct scientific research, including data analysis tools, simulation software, com-

putational pipelines, scientific frameworks, and domain-specific research libraries [1]. Repositories were considered research software if they demonstrated a clear scientific or research context through their descriptions, repository topics, README documentation, associated publications, or the affiliation of the owning organisation.

3.2.2 Collection Strategy

To obtain a diverse dataset, repositories were collected from ten research domains: bioinformatics, computational biology, astronomy, neuroscience, climate science, scientific computing, research software, cheminformatics, computational physics, and reproducible research. Each domain was queried using the GitHub Search API with the following pattern:

```
topic:<domain> stars:>20 archived:false
```

A minimum threshold of 20 GitHub stars was applied to filter out inactive or personal experimental repositories. Archived repositories were excluded as they are no longer actively maintained. The collection was capped at 20 repositories per topic to ensure balanced domain representation, yielding an initial dataset of 198 repositories.

3.2.3 Filtering and Cleaning

Following collection, repositories were automatically filtered according to predefined inclusion criteria. Forks, archived repositories, repositories whose most recent push occurred before 1 January 2023, and repositories whose names or descriptions contained predefined keywords — such as *awesome*, *tutorial*, *course*, *template*, *workshop*, *lecture*, *homework*, *assignment*, and *cheatsheet* — were excluded, as these typically represent curated lists, educational materials, or project templates rather than executable research software. These filtering steps reduced the initial dataset from 198 to 188 repositories.

3.2.4 Dataset Characteristics

The final dataset contains 188 research software repositories spanning 10 research domains and multiple programming language ecosystems. Python is the most common language (86 repositories, 46%), followed by C++ (29, 15%) and Jupyter Notebook (18, 10%). Additional languages include Julia, R, Rust, Java, Fortran, Go, and JavaScript, providing diversity beyond a single programming ecosystem. All repositories were publicly accessible on GitHub and had been actively maintained at the time of collection, whose latest repository activity occurred on or after 1 January 2023.

For each repository, the following data were collected via the GitHub API: repository metadata (name, owner, description, primary language, star count, topics, and

last push date), README file content, and the presence of FAIR-related repository files including license files, citation files, dependency specifications, structured metadata files, and contribution guidelines.

3.3 Ground Truth Construction

3.3.1 Purpose and Scope

To evaluate the accuracy of both the rule-based and LLM-based assessment components, a ground truth dataset was constructed through manual annotation of a subset of 30 repositories drawn from the full dataset. Manual annotations provide reference scores against which the automated assessments can be compared, enabling the computation of agreement metrics and the identification of systematic scoring errors.

3.3.2 Repository Selection

The 30 ground truth repositories were selected to ensure diversity across research domains and programming languages. The initial 10 repositories were annotated during the pilot phase of system development, primarily consisting of Python-based scientific software projects. Following supervisor feedback, 20 additional repositories were selected from the full dataset to broaden coverage across all 10 research domains and to include languages beyond Python, such as C++, Java, MATLAB, Fortran, Julia, Go, Rust, and Groovy. The combined ground truth set spans all 10 research domains represented in the full dataset.

3.3.3 Annotation Guidelines

Annotations were performed using a structured rubric developed specifically for this thesis, covering both the rule-based indicators (F1, F2, A1, I1, R1, R2) and the AI-based documentation quality dimensions (D1–D6). The rubric defines precise scoring criteria for each indicator and dimension, including edge case handling for non-standard license filenames, alternative citation file formats, and the treatment of external documentation links.

The annotation guidelines were developed iteratively during the pilot phase and refined through validation against automated scorer outputs. The final version, v1.4, clarified the distinction between structured citation metadata (F2) and explicit citation instructions (R2), and resolved ambiguities identified during ground truth annotation. All 30 repositories were annotated against the finalised v1.4 guidelines to ensure consistency.

3.3.4 Annotation Process

Each repository was annotated by the thesis author by examining the repository root file structure and README content. Rule-based indicators (F1–R2) were scored by checking for the presence of specific files and patterns in the README text. AI-based dimensions (D1–D6) were scored on a 0–2 scale by reading the README and assessing the clarity and completeness of installation instructions, usage guidance, purpose description, dependency information, examples, and contribution information.

Each annotation record includes the assigned score together with evidence notes justifying the decision. For the AI-based dimensions, annotations additionally record the annotator's confidence level (HIGH, MEDIUM, or LOW), reflecting the degree of ambiguity encountered when applying the rubric criteria.

3.3.5 Inter-Annotator Agreement

To assess annotation reliability, a second annotator was provided with the finalised v1.4 annotation guidelines, the list of 30 repositories with GitHub links, pre-fetched README files, FAIR-related file presence summaries, and blank annotation forms. The annotation package was designed to enable fully independent annotation without requiring access to the primary annotations or the GitHub repositories directly.

Upon completion of the independent annotation process, inter-annotator agreement was quantified using Cohen's weighted Kappa for the AI-based dimensions (D1–D6), where partial agreement is meaningful, and Cohen's unweighted Kappa for the binary rule-based indicators (F1–R2). Exact agreement was additionally reported to aid interpretation in cases where Kappa was undefined because of a lack of class variation.

Disagreements and anomalous results were subsequently reviewed against the final annotation guidelines and the collected repository evidence. This validation identified, among other cases, an inconsistency between the file-detection schema originally used for the ground-truth collection and the expanded schema used for the full dataset. The ground-truth file inventories and affected primary annotations were corrected where the repository evidence unambiguously supported the correction. The final agreement results are reported in Chapter 5.

3.4 Rule-Based Assessment

3.4.1 Overview

The rule-based assessment component evaluates each repository against six binary indicators derived from the FAIR4RS principles [1] and the FAIR-IMPACT research software metrics [2]. Each indicator checks for the presence of repository files or textual patterns that provide evidence for selected FAIR4RS indicators. The com-

ponent produces a score of 0 or 1 per indicator, with a maximum total score of 6.

3.4.2 Indicators

The six rule-based indicators are defined as follows:

F1 — Persistent Identifier Score 1 if evidence of a persistent identifier (e.g., DOI, Zenodo, or Software Heritage identifier) is detected in the repository documentation or metadata. Detected patterns include doi.org URLs, Zenodo badges, Software Heritage identifiers, and DOI badge URL formats.

F2 — Structured Metadata Score 1 if a structured metadata file is present in the repository root. Accepted metadata files are `CITATION.cff` and `codemeta.json`, both of which provide machine-readable software metadata.

A1 — License Score 1 if a license file is present in the repository root or if a license is declared in the GitHub repository metadata. Accepted license filenames include LICENSE, LICENSE.md, LICENSE.txt, LICENSE.rst, COPYING, and COPYING.txt.

I1 — Dependency Specification Score 1 if a dependency specification file is present in the repository root. Accepted files cover multiple language ecosystems, including `requirements.txt`, `pyproject.toml`, `environment.yml`, `setup.py`, `Cargo.toml`, `go.mod`, `pom.xml`, `Project.toml`, and `package.json`.

R1 — README Present Score 1 if a README file is present in the repository root. Accepted filenames include README.md, README.rst, README.txt, and README.

R2 — Citation Information Score 1 if explicit citation instructions are present in the repository. This includes dedicated citation files (CITATION.md, CITATION.bib, CITATION.txt, CITATION, CITATION.cff) and explicit citation instructions in the README text, such as phrases containing "please cite", "how to cite", "if you use … cite", or "we would appreciate citations".

3.4.3 Implementation

Repository data were collected in a preprocessing step and stored locally as JSON files. The scorer operates exclusively on these locally stored artefacts rather than querying the GitHub API during scoring, ensuring deterministic and reproducible execution. For each repository, the scorer reads the FAIR file presence record (`fair_file_checks.json`), the repository metadata (`metadata.json`), and the README file. File presence checks are performed against the stored JSON record, while F1 and R2 additionally apply regular expression pattern matching against the README text.

The scorer was validated against the 30 manually annotated ground truth repositories. Initial validation revealed discrepancies in several indicators. Detection logic for F1 and R2 was refined to cover additional DOI representations, citation file formats, and citation instruction phrasings identified during manual validation.

A subsequent consistency check also identified schema drift between the collector originally used for the ground-truth repositories and the collector used for the full dataset. The earlier collector did not check several language-specific dependency and build files, including `Cargo.toml`, `go.mod`, `pom.xml`, `Project.toml`, and `build.gradle`. The ground-truth file inventories were therefore aligned with the expanded schema used for the full dataset, and affected annotations were corrected where the repository evidence unambiguously satisfied the final rubric. Additional manual annotation inconsistencies identified during validation were likewise corrected against the finalised guidelines. The final validation results, including precision, recall, and F1-score per indicator, are reported in Chapter 5.

3.4.4 Output

The rule-based scorer produces a CSV file containing one row per repository with individual indicator scores and the aggregated rule-based score. The scorer was subsequently applied to all 188 repositories in the dataset, producing the rule-based assessment results used in the evaluation presented in Chapter 5.

3.5 LLM-Based Assessment

3.5.1 Overview

The LLM-based assessment component evaluates the quality and completeness of repository README documentation against six documentation quality dimensions derived from FAIR4RS reusability indicators [1] and the software documentation quality framework proposed by Treude et al. [10]. Unlike the rule-based component, which checks for the presence of files and patterns, the LLM-based component assesses the content and clarity of documentation that a potential user would encounter when attempting to reuse the software.

3.5.2 Documentation Quality Dimensions

Six documentation quality dimensions are evaluated, each scored on a 0–2 scale:

D1 — Installation Clarity Assesses whether a new user can install the software from the README alone. Score 2 if complete and actionable installation steps are provided; score 1 if installation information exists but is incomplete or links externally; score 0 if no installation instructions are present.

D2 — Usage Clarity Assesses whether a new user can understand how to run or use the software from the README. Score 2 if a concrete command, code snippet, or executable example is present; score 1 if external documentation is clearly linked but no inline example appears; score 0 if no usage information is present.

D3 — Purpose Description Assesses whether the README clearly explains what the software does and its intended domain or audience. Score 2 if the purpose is clearly described; score 1 if a description is present but vague or incomplete; score 0 if no description is present.

D4 — Dependency Description Assesses whether the README describes the environment or dependencies required to use the software. Score 2 if dependencies are explicitly listed in human-readable form; score 1 if dependency information is only partially provided; score 0 if no dependency information is present.

D5 — Examples and Tutorials Assesses whether the repository provides concrete examples that support reuse. Score 2 if the README contains a meaningful usage example demonstrating software use. References to external example galleries or tutorials support score 1 unless their content is included in the supplied README evidence. Score 0 if no examples or tutorials are mentioned.

D6 — Contribution Information Assesses whether the README explains how to contribute to or get support for the software. Score 2 if contribution guidance is provided together with at least one community or support channel; score 1 if contribution or support information is only partially present; score 0 if neither is available.

3.5.3 Model Selection

The open-source Llama 3.1 8B model was selected because it can be executed locally through Ollama, ensuring reproducibility and eliminating dependence on commercial APIs. Running the model locally also avoids API usage costs and enables repeated experiments under identical inference settings. Temperature was set to 0 to maximise output determinism across repeated runs.

3.5.4 Prompt Design

The LLM is provided with the repository name and README content as input and instructed to score each dimension according to the defined rubric. The prompt specifies that the model must evaluate only evidence explicitly present in the README text, and must not assume content exists behind external links. Each score must be

accompanied by a confidence level (HIGH, MEDIUM, or LOW) and a one-sentence reasoning statement citing specific README evidence.

The final prompt was developed through iterative experimentation on the pilot repository set, with refinements improving the specificity of dimension definitions, particularly for D5, where generic documentation links were being incorrectly treated as usage examples.

3.5.5 Implementation

The LLM-based scorer uses the Ollama HTTP API with structured JSON output enforced through a JSON schema supplied to the API, ensuring that every response conforms to the expected structure and that scores are restricted to the permitted values (0, 1, or 2).

Each repository README is provided to the model in full. READMEs exceeding 12,000 characters are truncated prior to inference to remain within practical context limits. During the experiments, this affected one repository in the ground truth dataset (Ouranosinc/xclim). The scorer implements a retry mechanism, attempting a second API call if the first response fails validation. Raw model outputs and parsed JSON results are stored separately to support replication and later stability analysis.

Confidence values and reasoning statements were retained to support qualitative error analysis but were not included in the quantitative scoring metrics.

3.5.6 External Link Policy

The LLM receives only the README text as input and cannot access external documentation linked from the README. Consequently, linked resources not reproduced in the README are treated as partial evidence (score 1) rather than complete evidence (score 2). The same README-only interpretation was applied during manual annotation to ensure a fair comparison between human and automated assessment. This design choice is documented as a limitation and is considered when interpreting differences between LLM and manual annotations.

3.5.7 Output

The LLM scorer produces a CSV file containing one row per repository with scores, confidence levels, and reasoning for each of the six dimensions, together with the model name and a timestamp. The scorer was applied to all 30 ground truth repositories, producing the LLM-based assessment results used in the evaluation presented in Chapter 5.

3.6 Repository FAIRness Profile

The hybrid assessment framework combines the outputs of the rule-based and LLM-based components into a unified FAIRness profile for each repository. Rather than aggregating the two components into a single numerical score, the framework preserves them as complementary assessment dimensions. This design maintains the interpretability of both structural FAIR indicators and qualitative documentation quality, allowing users to identify strengths and weaknesses across different aspects of software FAIRness.

The rule-based component contributes six binary indicator scores (F1–R2) covering structural metadata compliance, together with an aggregated rule-based total. The LLM-based component contributes six documentation quality scores (D1–D6) on a 0–2 ordinal scale, each accompanied by a confidence level and a reasoning statement. Both sets of scores are combined into a single structured record per repository, stored as a row in a CSV output file.

This parallel representation provides more actionable feedback than a single aggregated score would allow. A repository may score highly on structural metadata indicators while exhibiting weak documentation quality, or conversely may have well-written documentation but lack persistent identifiers or structured citation metadata. Presenting both dimensions separately makes these patterns visible and directly supports the identification of specific FAIRness gaps.

The resulting hybrid FAIRness profiles constitute the primary output of the proposed framework and are used as the basis for the evaluation presented in Chapter 5.

3.7 Evaluation Strategy

The evaluation strategy assesses the accuracy, reliability, and added value of the proposed hybrid assessment framework through both quantitative and qualitative methods. The evaluation is structured around the three research questions defined in Chapter 1 and is conducted against the 30-repository ground truth dataset described in Section 3.3.

3.7.1 Ground Truth Comparison

The primary evaluation compares the automated scores produced by both the rule-based and LLM-based components against the manual ground truth annotations. For the rule-based indicators (F1–R2), which are binary, performance against the manual ground truth is evaluated using precision, recall, and F1-score. Exact agreement and Cohen's unweighted Kappa are additionally reported to characterise agreement between assessments. For the LLM-based dimensions (D1–D6), which use a 0–2 ordinal scale, agreement is measured using exact match accuracy, Mean Absolute Error (MAE), and Cohen's weighted Kappa per dimension, where partial agreement is given proportional credit.

3.7.2 Inter-Annotator Agreement

To assess the reliability of the manual ground truth annotations, inter-annotator agreement between the primary annotator and the second annotator was quantified using Cohen's unweighted Kappa for binary indicators and Cohen's weighted Kappa for ordinal dimensions. Exact agreement was also reported, particularly because Kappa is undefined when both annotators assign only a single class. These results establish the reliability of the manual reference annotations and contextualise the agreement observed between the automated assessments and the ground truth.

3.7.3 Baseline Comparison

The proposed approach is compared with existing FAIR assessment tools. A direct comparison with howfairis is performed on the ground truth repositories, examining indicator coverage, agreements and disagreements, and FAIRness aspects not represented by the existing tool. F-UJI is separately evaluated in a pilot experiment by submitting plain GitHub repository URLs directly, to investigate whether a dataset-oriented FAIR assessment tool can serve as a meaningful quantitative baseline when applied to research software repositories without a mediating dataset record.

3.7.4 Qualitative Error Analysis

In addition to quantitative agreement metrics, representative cases of agreement and disagreement between the automated assessments and the manual annotations are analysed. The confidence levels and reasoning statements produced by the LLM are used to help interpret discrepancies and identify common sources of scoring differences.

3.7.5 Confidence Calibration

The usefulness of the LLM's self-reported confidence labels is evaluated by comparing prediction correctness across HIGH, MEDIUM, and LOW confidence outputs. Agreement with the primary human annotations is examined both across all documentation dimensions and within individual dimensions where sufficient observations are available. This analysis assesses whether confidence can be interpreted as an indicator of prediction reliability or should instead be treated only as contextual information.

3.7.6 README Length and Verbosity Bias

To investigate whether LLM scoring errors are associated with README length, the difference between the LLM and human documentation scores is compared with the effective README length supplied to the model. README length is measured after applying the same 12,000-character input limit used during inference.

Spearman's rank correlation is computed for the aggregate documentation score and separately for D1–D6. This analysis tests whether longer README files are systematically associated with higher LLM scores independently of the corresponding human assessments.

4 Implementation

5 Evaluation and Results

5.1 Experimental Setup

This chapter presents the quantitative evaluation of the proposed hybrid FAIRness assessment framework. The results assess the reliability of the framework and compare its outputs against manually annotated ground truth and an existing baseline approach.

5.1.1 Dataset

The full dataset comprises 188 research software repositories collected from GitHub across 10 research domains, as described in Chapter 3. A subset of 30 repositories was manually annotated to serve as the ground truth for quantitative evaluation. The manually annotated repositories form a representative subset of the complete dataset and were used exclusively for quantitative evaluation. The ground truth set was selected to ensure diversity across research domains and programming languages, covering all 10 domains represented in the full dataset and including repositories implemented in Python, C++, Java, MATLAB, Fortran, Julia, Go, Rust, and Groovy.

5.1.2 Evaluation Metrics

The following metrics are used throughout this chapter:

Exact Match Accuracy The proportion of repositories for which the automated score exactly matches the manual annotation. Reported as a percentage.

Mean Absolute Error (MAE) The average absolute difference between the automated assessment and the corresponding manual annotation, computed per dimension across all 30 ground truth repositories. Applied to the ordinal LLM-based dimensions (D1–D6).

Cohen's Unweighted Kappa Used for binary rule-based indicators (F1–R2), where any disagreement is treated equally regardless of magnitude.

Cohen's Linearly Weighted Kappa Used for ordinal AI-based dimensions (D1–D6), where partial agreement is given proportional credit. A disagreement of 1 point is penalised less than a disagreement of 2 points.

Kappa values are interpreted following the scale proposed by Landis and Koch [7]: values below 0.20 are considered slight, 0.21–0.40 fair, 0.41–0.60 moderate, 0.61–0.80 substantial, and above 0.80 almost perfect agreement.

5.1.3 Models and Tools

The LLM-based component uses the `llama3.1:8b` model executed locally via the Ollama HTTP API at temperature 0. The model was selected because it can be executed locally without external API calls, ensuring reproducibility and avoiding data transfer outside the experimental environment. Structured JSON output is enforced through a schema to constrain scores to the permitted values (0, 1, or 2). The rule-based component operates deterministically on locally stored repository snapshots.

The howfairis tool is used as a software-oriented baseline comparison. F-UJI, an automated FAIR assessment tool designed primarily for research data objects, is additionally used in a pilot experiment to investigate the applicability of an external, data-oriented FAIR assessment baseline to research software repositories.

5.1.4 Experimental Environment

The evaluation pipeline was implemented in Python and executed locally on Windows using Ollama for LLM inference. The LLM inference was performed using the `llama3.1:8b` model (4.9 GB). All scripts, data, and results are stored in the project repository to support reproducibility.

5.1.5 Evaluation Procedure

The evaluation was conducted in the following order:

1. The thesis author manually annotated 30 ground truth repositories against the finalised annotation guidelines (v1.4).
2. A second annotator independently annotated the same 30 repositories using the same guidelines.
3. Inter-annotator agreement was computed between the two human annotators to validate the reliability of the ground truth.
4. The LLM scorer was applied to the same 30 repositories and the outputs were compared against both annotators.
5. The rule-based scorer was applied to all 188 repositories to produce a complete set of structural FAIR indicator scores.
6. Hybrid FAIRness profiles were generated by combining rule-based and LLM scores for the 30 ground truth repositories.

7. The howfairis baseline was executed on the 30 ground truth repositories and compared against the proposed framework.

8. F-UJI was applied directly to a subset of five ground truth repositories to investigate its applicability as an external data-oriented FAIRness assessment baseline for research software.

5.2 Ground Truth Validation

5.2.1 Annotation Process

Manual annotations were created by the thesis author for all 30 ground truth repositories following the finalised annotation guidelines (v1.4), as described in Chapter 3. Each repository was annotated by examining the repository root file structure and README content. Rule-based indicators (F1–R2) were assigned binary scores according to the annotation guidelines based on evidence obtained from the repository structure and README content. AI-based dimensions (D1–D6) were scored on a 0–2 ordinal scale based on the clarity and completeness of the README documentation.

To assess annotation reliability, a second annotator independently annotated the same 30 repositories using identical guidelines and pre-fetched repository data, without access to the primary annotations.

5.2.2 Inter-Annotator Agreement

Inter-annotator agreement between the two annotators was quantified using Cohen's unweighted Kappa for binary rule-based indicators and Cohen's linearly weighted Kappa for ordinal AI-based dimensions. Table 5.1 presents the results.

Overall, the agreement results indicate that both the binary rule-based indicators and the ordinal documentation quality dimensions can be annotated consistently using the proposed guidelines.

The AI-based dimensions achieved a macro-average weighted Kappa of 0.709, indicating substantial overall agreement between annotators. The strongest agreement was observed for D6 (contribution information, κ=0.849) and D3 (purpose description, κ=0.789). The weakest agreement was observed for D1 (installation clarity, κ=0.539) and D5 (examples and tutorials, κ=0.585), suggesting greater ambiguity when assessing partially documented functionality or documentation provided through external links.

For the rule-based indicators, F2 achieved perfect agreement, while F1, I1, and R2 showed substantial agreement. Two indicators are mathematically undefined: R1 (README present), because all 30 repositories have a README file, and A1 (license), because both annotators agreed on the presence of a license for all 30 repositories following a correction described below. Both indicators demonstrate consistent annotation in practice, as reflected by their 100% exact agreement.

Table 5.1: Inter-annotator agreement for rule-based indicators and AI-based dimensions (n=30).

Indicator/Dimension	Type	κ	Exact	Interpretation
F1 — Persistent identifier	Binary	0.710	90.0%	Substantial
F2 — Structured metadata	Binary	1.000	100.0%	Almost perfect
A1 — License	Binary	undef.	100.0%	Single class
I1 — Dependency file	Binary	0.634	93.3%	Substantial
R1 — README present	Binary	undef.	100.0%	Single class
R2 — Citation information	Binary	0.658	83.3%	Substantial
Rule macro-avg		**0.750**		
D1 — Installation	Ordinal	0.539	70.0%	Moderate
D2 — Usage	Ordinal	0.747	83.3%	Substantial
D3 — Purpose	Ordinal	0.789	96.7%	Substantial
D4 — Dependencies	Ordinal	0.743	80.0%	Substantial
D5 — Examples	Ordinal	0.585	70.0%	Moderate
D6 — Contribution	Ordinal	0.849	90.0%	Almost perfect
AI macro-avg		**0.709**		

The rule-based macro-average of 0.750 excludes the two undefined values (A1 and R1).

I1 (dependency file present) initially showed disagreement on seven repositories, predominantly projects using non-Python language ecosystems (C++, Java, Go, Rust, Groovy, Julia), following the pattern that the primary annotator assigned a score of 0 while the second annotator assigned a score of 1. Investigation traced this to an incomplete file-detection schema used during ground-truth collection: the collector script for the 30 ground-truth repositories predated the expanded dependency-file list (including `go.mod`, `Cargo.toml`, `pom.xml`, `build.gradle`, and `Project.toml`) later adopted for the full 188-repository collection, and consequently could not detect language-specific dependency files for non-Python projects. Five of the seven affected repositories were confirmed via the corrected file inventory to genuinely contain a dependency specification file, and the corresponding primary annotations were corrected accordingly. Following this correction, I1 agreement improved from κ=0.286 (fair) to κ=0.634 (substantial).

A1 (license) was also corrected during validation: one repository (`rust-ml/linfa`) had been annotated as lacking a license, despite both the GitHub repository metadata and the README text explicitly identifying an Apache-2.0 license. This annotation was corrected, resulting in full agreement between annotators for A1.

These inter-annotator agreement results demonstrate sufficient reliability of the manual annotations to serve as the ground truth for the subsequent evaluation.

5.3 Rule-Based Assessment Results

The rule-based scorer was applied to all 188 repositories in the full dataset, producing binary scores for each of the six FAIR indicators (F1–R2). This section presents the distribution of rule-based assessment results across the full dataset and evaluates the rule-based component against the manually annotated ground truth repositories.

5.3.1 Full Dataset Results

Table 5.2 summarises the frequency of positive scores for each indicator across all 188 repositories.

Table 5.2: Rule-based indicator frequencies across the full dataset (n=188).

Indicator	Description	Positive	Percentage
F1	Persistent identifier	95	51%
F2	Software metadata	58	31%
A1	License	182	97%
I1	Dependency file	123	65%
R1	README present	188	100%
R2	Citation information	79	42%

All 188 repositories contain a README file (R1=100%), and 97% have a license file (A1=97%). In contrast, dedicated software metadata files such as `CITATION.cff` or `codemeta.json` are present in only 31% of repositories (F2), and persistent identifiers such as DOIs or Zenodo badges are present in only 51% (F1). Citation information is available in 42% of repositories (R2), suggesting that fewer than half of the evaluated repositories provide explicit guidance on how to cite the software. Overall, the results indicate that basic repository documentation practices, such as providing README and license files, are widespread, whereas richer FAIR-related metadata and citation support remain considerably less common.

The mean rule-based score across the full dataset was 3.86 out of a maximum possible score of 6. The score distribution is shown in Table 5.3.

Table 5.3: Distribution of rule-based total scores across the full dataset (n=188).

Score	Count	Percentage
6/6	15	8%
5/6	43	23%
4/6	61	32%
3/6	42	22%
2/6	23	12%
1/6	4	2%

The score distribution spans the full range from 1 to 6, indicating substantial variability in structural FAIR compliance across the dataset. Only 15 repositories (8%) achieve full compliance across all six rule-based indicators, while 27 repositories (14%) score 2 or below, indicating significant metadata gaps.

5.3.2 Validation Against Ground Truth

The rule-based scorer was validated against the final manual annotations for the 30 ground truth repositories. During validation, differences between automated predictions and manual annotations were inspected individually and resolved according to the finalised annotation guidelines (v1.4).

This process identified both implementation-level detection gaps and manual annotation inconsistencies, particularly affecting the citation (R2), persistent identifier (F1), and license (A1) indicators. The rule-based detection logic was refined to recognise additional valid citation and persistent identifier patterns, including `CITATION.cff` files, generic DOI representations, and explicit citation wording. Manual annotations that were inconsistent with the final rubric were corrected and documented in the corresponding repository annotation files.

After these refinements, the final rule-based scorer achieved exact agreement with the final ground truth annotations across all 30 repositories. Precision, recall, and F1-score were 1.000 for each of the six binary indicators (F1–R2), as shown in Table 5.4.

Table 5.4: Rule-based validation against manual ground truth (n=30).

Indicator	**Precision**	**Recall**	**F1-score**
F1 — Persistent identifier	1.000	1.000	1.000
F2 — Software metadata	1.000	1.000	1.000
A1 — License	1.000	1.000	1.000
I1 — Dependency file	1.000	1.000	1.000
R1 — README present	1.000	1.000	1.000
R2 — Citation information	1.000	1.000	1.000

The final results demonstrate that the rule-based implementation faithfully reproduces the operational definitions encoded in the manual annotation rubric for the evaluated ground truth set. This result should be interpreted as confirmation that the rule-based implementation correctly encodes the operational definitions specified in the finalised annotation guidelines, rather than as an estimate of accuracy on unseen repositories: both the scorer and portions of the ground truth were iteratively refined against this same validation set during development.

5.4 LLM-Based Assessment Results

The LLM-based scorer was applied to all 30 ground truth repositories using the `llama3.1:8b` model at temperature 0. This section presents the agreement between LLM-generated documentation quality scores and manual annotations from both annotators, and identifies systematic patterns in LLM scoring behaviour.

5.4.1 LLM vs Primary Annotator

Table 5.5 presents the agreement between LLM scores and the primary annotator across all six documentation quality dimensions.

Table 5.5: LLM vs primary annotator agreement (n=30).

Dimension	κ	**Exact**	**MAE**	**Interpretation**
D1 — Installation	0.605	70.0%	0.333	Moderate
D2 — Usage	0.274	46.7%	0.567	Fair
D3 — Purpose	−0.071	86.7%	0.133	Prevalence effect
D4 — Dependencies	0.480	60.0%	0.433	Moderate
D5 — Examples	0.192	36.7%	0.700	Slight
D6 — Contribution	0.494	63.3%	0.400	Moderate
Macro-avg	**0.329**			Fair

5.4.2 LLM vs Second Annotator

Table 5.6 presents the agreement between LLM scores and the second annotator.

Table 5.6: LLM vs second annotator agreement (n=30).

Dimension	κ	**Exact**	**MAE**	**Interpretation**
D1 — Installation	0.480	70.0%	0.433	Moderate
D2 — Usage	0.276	50.0%	0.600	Fair
D3 — Purpose	−0.056	86.7%	0.167	Prevalence effect
D4 — Dependencies	0.338	53.3%	0.567	Fair
D5 — Examples	0.156	30.0%	0.867	Slight
D6 — Contribution	0.454	60.0%	0.433	Moderate
Macro-avg	**0.275**			Fair

5.4.3 Systematic Scoring Patterns

Analysis of the direction of disagreements between LLM and human scores reveals a consistent pattern: the LLM assigned lower scores than human annotators more

frequently than higher scores across almost all dimensions and both annotators. Table 5.7 summarises the direction of disagreements for the primary annotator comparison.

Table 5.7: Direction of LLM scoring disagreements relative to primary annotator (n=30). LLM lower = LLM score < human score; LLM higher = LLM score > human score.

Dimension	LLM lower	LLM higher
D1 — Installation	8	1
D2 — Usage	14	2
D3 — Purpose	2	2
D4 — Dependencies	8	4
D5 — Examples	17	2
D6 — Contribution	9	2

The most pronounced underscoring occurs in D5 (examples and tutorials, 17 cases lower vs 2 higher) and D2 (usage clarity, 14 cases lower vs 2 higher). These dimensions are most affected by the external link policy adopted in this study: the LLM evaluates only README text and cannot access external documentation linked from it. Repositories that reference tutorials, usage guides, or example galleries via external links receive lower LLM scores than human annotators who take the existence of those resources into account.

D3 (purpose description) shows a negative Kappa (-0.071) despite 86.7% exact agreement, reflecting a prevalence effect: 28 of 30 repositories received a human score of 2 for purpose description, leaving insufficient variation for meaningful Kappa computation.

D1 (installation clarity) achieved the strongest agreement with the primary annotator (κ=0.605), consistent with the observation that installation instructions are typically provided inline in README files and are therefore fully accessible to the LLM.

5.4.4 Comparison with Inter-Annotator Agreement

Table 5.8 compares LLM-human agreement with human-human agreement.

Table 5.8: Agreement summary: human-human vs LLM-human (macro-avg weighted κ, AI dimensions only).

Comparison	Macro-avg κ
Human A1 vs Human A2	0.709
LLM vs Human A1	0.329
LLM vs Human A2	0.275

Human-human agreement (κ=0.709) substantially exceeds LLM-human agreement (κ=0.329 and 0.275), confirming that the LLM assessment is less reliable than human annotation for this task. The consistency of the pattern across both annotator comparisons suggests that the findings reflect genuine LLM behaviour rather than idiosyncrasies of one annotator's scoring style.

5.4.5 Confidence Calibration

The LLM scorer additionally reports a self-assessed confidence level (HIGH, MEDIUM, or LOW) for each documentation quality score. To evaluate whether these confidence labels correspond to actual agreement with the human ground truth, the 180 individual dimension-level predictions (30 repositories × 6 dimensions) were grouped by reported confidence level. Exact agreement, mean absolute error (MAE), and linearly weighted Cohen's Kappa were then computed within each confidence group.

Table 5.9 summarises the results across all six documentation dimensions.

Table 5.9: LLM confidence calibration against the primary human annotations across all documentation dimensions.

Confidence	**n**	**Exact**	**MAE**	κ
HIGH	113	56.6%	0.460	0.520
MEDIUM	60	71.7%	0.283	0.478
LOW	7	28.6%	1.143	−0.120

The reported confidence levels were not monotonically associated with prediction accuracy. Predictions labelled MEDIUM achieved the highest exact agreement (71.7%) and the lowest MAE (0.283), whereas HIGH-confidence predictions achieved lower exact agreement (56.6%) and a higher MAE (0.460). LOW-confidence predictions performed worst, with 28.6% exact agreement and an MAE of 1.143, although this group contained only seven predictions and should therefore be interpreted cautiously.

The dimension-level results further show that confidence reliability varies across assessment dimensions. For example, HIGH-confidence predictions for D3 (purpose) achieved 94.4% exact agreement, whereas HIGH-confidence predictions for D5 (examples) achieved only 29.4%. Similarly, MEDIUM-confidence predictions outperformed HIGH-confidence predictions in exact agreement for D1, D2, D4, and D5. This pattern is consistent with the systematic scoring behaviour identified in Section 5.4.3, particularly for D2 and D5. In these dimensions, the model frequently assigned lower scores when relevant usage guidance or examples were provided through external documentation rather than directly in the README. The poor accuracy of HIGH-confidence predictions in these dimensions therefore suggests that the model can be confident even when its assessment differs from the human interpretation of the rubric.

These results indicate that the model's self-reported confidence provides some useful information at the low-confidence extreme but is not a consistently calibrated indicator of prediction correctness. In particular, a HIGH confidence label should not be interpreted as guaranteeing stronger agreement with human assessment. Confidence should therefore be retained as contextual information in the hybrid FAIRness profile rather than treated as a quantitative reliability estimate.

5.4.6 README Length and Verbosity Bias

To investigate whether the LLM systematically assigned higher scores to repositories with longer README files, README length was compared with the difference between the LLM and human assessment scores. For each repository, the score difference was calculated as the LLM total minus the corresponding human total. README length was measured using the effective number of words available to the LLM after applying the same 12,000-character input limit used during assessment.

Spearman's rank correlation showed a weak negative and non-significant relationship between README length and scoring difference ($\rho = -0.169$, $p = 0.371$, $n = 30$). Thus, longer README files were not associated with systematically inflated LLM scores in the evaluated sample.

The same analysis was performed separately for each documentation dimension. No statistically significant correlation was observed for D1–D6, with coefficients ranging from $\rho = -0.180$ for dependency documentation (D4) to $\rho = 0.060$ for installation documentation (D1). Notably, D2 (usage) and D5 (examples) — the dimensions identified in Section 5.4.3 as most affected by systematic underscoring due to the external link policy — showed weak negative correlations ($\rho = -0.110$ and $\rho = -0.135$ respectively), the opposite direction from what a verbosity bias would predict.These results provide no evidence that README length offers a competing explanation for the underscoring observed in those dimensions. These results provide no evidence of a systematic verbosity bias within the evaluated ground-truth set.

Only one of the 30 README files exceeded the 12,000-character input limit used by the LLM assessment pipeline. Consequently, truncation was unlikely to substantially influence the observed relationship between README length and scoring error. Nevertheless, the relatively small ground-truth sample means that the absence of a significant correlation should not be interpreted as evidence that verbosity bias cannot occur on a broader repository population.

5.5 Hybrid FAIRness Profile Analysis

The hybrid FAIRness profile combines the rule-based indicators (F1–R2) and LLM-based documentation quality scores (D1–D6) into a single structured record per repository. This section analyses the combined profiles for the 30 ground truth repositories and examines the relationship between the two assessment dimensions.

5.5.1 Score Distributions

Table 5.10 presents the mean scores for both components across the 30 ground truth repositories.

Table 5.10: Mean scores for rule-based and LLM-based components across the 30 ground truth repositories.

Component	Mean Score	Maximum
Rule-based total	4.30	6
LLM total	6.70	12

The mean rule-based score across the 30 ground truth repositories was 4.30 out of 6, slightly higher than the mean across the full 188-repository dataset (3.86). The mean LLM total was 6.70 out of 12.

5.5.2 Relationship Between Rule-Based and LLM Scores

To examine whether repositories that score highly on structural FAIR indicators also score highly on documentation quality, Spearman's rank correlation was computed between the rule-based total and the LLM total across the 30 ground truth repositories. The result ($\rho = 0.175$, $p = 0.356$) indicates a weak positive association that was not statistically significant.

No clear monotonic association was therefore detected between the two components in the evaluated sample. This weak relationship supports treating structural FAIR compliance and documentation quality as complementary assessment dimensions. A repository may achieve a high rule-based score through the presence of formal metadata files while providing limited inline documentation, or conversely may offer well-written README content while lacking persistent identifiers or structured citation metadata. The hybrid profile captures both dimensions simultaneously, providing a more complete picture than either component alone.

5.5.3 Complementary Assessment Dimensions

The evaluated repositories exhibited complementary assessment patterns. Repositories with high rule-based scores but comparatively low LLM scores possess strong structural FAIR metadata while providing limited inline documentation. Conversely, other repositories achieve higher documentation scores despite lacking formal metadata such as `CITATION.cff` files or persistent identifiers.

By presenting both dimensions in parallel, the hybrid profile allows users to identify specific FAIRness gaps and prioritise improvements accordingly, rather than receiving a single aggregated score that obscures the nature of any deficiencies.

5.5.4 Profile Interpretation

The hybrid FAIRness profile provides a more informative basis for repository assessment than either component alone. A repository with high scores in both components demonstrates both structural FAIR compliance and high-quality documentation. Conversely, a repository with a high rule-based score but a low LLM score may be formally compliant while remaining difficult to reuse in practice. Similarly, a repository with lower rule-based scores but higher LLM scores may provide excellent documentation while lacking the machine-readable metadata required for automated discovery and citation.

5.6 Baseline Comparison

This section compares the proposed hybrid assessment framework with two existing FAIR assessment approaches. The software-oriented howfairis tool is evaluated across all 30 ground truth repositories and provides a direct comparison of assessment scope. F-UJI is examined separately through a five-repository pilot to determine whether a data-oriented FAIR assessment tool can provide a meaningful quantitative baseline when applied directly to research software repositories hosted on GitHub.

5.6.1 F-UJI Baseline Applicability

Before using F-UJI as a quantitative baseline, its applicability to research software repositories was investigated. F-UJI is designed primarily for assessing the FAIRness of research data objects. The pilot therefore examined whether it could meaningfully differentiate research software repositories when provided with plain GitHub repository URLs.

Five ground truth repositories with different rule-based and documentation quality profiles were submitted directly to F-UJI (version 3.5.1, metrics v0.8): `astropy/astropy` and `sunpy/sunpy` with rule-based scores of 6/6, `gonum/gonum` with a rule-based score of 2/6 and an LLM documentation score of 5/12, `yt-project/yt` with scores of 4/6 and 11/12, respectively, and `scipy/scipy` with scores of 5/6 and 8/12. Table 5.11 presents the results.

Despite rule-based scores ranging from 2/6 to 6/6 and LLM documentation scores ranging from 5/12 to 11/12, all five repositories received the same F-UJI score of 42.31% (11/26) and the same FAIR category breakdown. F-UJI also reported that the evaluated resources did not identify themselves as datasets, with the detected resource types including `softwaresourcecode` and `object`.

Within this pilot, F-UJI therefore provided no differentiation between repositories with substantially different software-specific FAIRness profiles. Inspection of the metric-level outputs suggests that several F-UJI metrics rely on metadata representations that are not directly aligned with repository-level software evidence.

Table 5.11: F-UJI assessment results compared with the proposed framework's rule-based and LLM documentation scores (n=5).

Repository	Rule	LLM	F-UJI	F	A	I	R
`astropy/astropy`	6/6	7/12	42.31%	42.86%	42.86%	33.33%	50.00%
`sunpy/sunpy`	6/6	9/12	42.31%	42.86%	42.86%	33.33%	50.00%
`gonum/gonum`	2/6	5/12	42.31%	42.86%	42.86%	33.33%	50.00%
`yt-project/yt`	4/6	11/12	42.31%	42.86%	42.86%	33.33%	50.00%
`scipy/scipy`	5/6	8/12	42.31%	42.86%	42.86%	33.33%	50.00%

For example, the license metric for `astropy/astropy` failed despite the repository explicitly identifying a BSD 3-clause license.

Given the identical results across the five varied repositories and F-UJI's resource-type warning, a quantitative comparison across all 30 ground truth repositories was not pursued. The pilot is instead treated as an applicability finding: when applied directly to the tested GitHub repository URLs, F-UJI was not a sufficiently discriminative quantitative baseline for the software-oriented assessment performed in this study.

5.6.2 howfairis Assessment Results

The howfairis tool was applied to all 30 ground truth repositories. howfairis evaluates five binary indicators: open repository, license, software registry, citation, and checklist (presence of a fair-software.eu badge). Table 5.12 summarises the score distribution.

Table 5.12: howfairis score distribution across the 30 ground truth repositories.

Score	Count	Percentage
5/5	1	3%
4/5	15	50%
3/5	11	37%
2/5	3	10%

Only one repository (`Ouranosinc/xclim`) achieved a full score of 5/5. The checklist indicator, which requires the presence of a fair-software.eu badge in the repository README, was satisfied by only one repository in the ground truth set. Consequently, this indicator provided little differentiation between repositories in the evaluated sample. Both the open repository and license indicators passed for all or nearly all repositories, consistent with the rule-based results reported in Section 5.3.

5.6.3 Scope Comparison

Table 5.13 compares the assessment dimensions covered by howfairis, the rule-based component, and the LLM-based component of the proposed framework. F-UJI is not included in this dimension-by-dimension table because the pilot demonstrated that its data-oriented metrics were not directly comparable to the software-specific dimensions evaluated by the proposed framework.

Table 5.13: Comparison of assessment scope across howfairis, the rule-based component, and the LLM-based component.

Dimension	howfairis	Rule-based	LLM-based
Open repository	✓	–	–
License	✓	✓(A1)	–
Software registry	✓	–	–
Citation metadata	✓	✓(F2, R2)	–
Checklist badge	✓	–	–
Persistent identifier	–	✓(F1)	–
Dependency file	–	✓(I1)	–
Installation clarity	–	–	✓(D1)
Usage clarity	–	–	✓(D2)
Purpose description	–	–	✓(D3)
Dependency description	–	–	✓(D4)
Examples/tutorials	–	–	✓(D5)
Contribution info	–	–	✓(D6)

The comparison shows that howfairis and the proposed framework cover largely complementary assessment dimensions. Both approaches assess licensing and citation-related metadata, which represents the primary overlap between them. The proposed framework additionally evaluates persistent identifiers and dependency specifications through its rule-based component, as well as six documentation quality dimensions through its LLM-based component. These aspects are not assessed by howfairis.

During the evaluation, the software registry indicator appeared to depend on repository badges displayed in the README rather than direct verification of registry entries. As a result, repositories available through a software registry but lacking the corresponding badge received a negative registry score.

5.7 Summary of Findings

This section summarises the key findings from the evaluation and addresses the research questions defined in Chapter 1.

5.7.1 Ground Truth Reliability

Inter-annotator agreement between the two human annotators was substantial for both the AI-based documentation quality dimensions (macro-average κ=0.709) and the rule-based indicators (macro-average κ=0.750, excluding the undefined A1 and R1 values). These results demonstrate sufficient reliability of the manual annotations to serve as ground truth for the subsequent evaluation. An initial moderate agreement on I1 (dependency file, κ=0.286) was traced to an incomplete file-detection schema used during ground-truth collection rather than genuine annotator disagreement; following correction, I1 agreement improved to κ=0.634 (substantial).

5.7.2 Rule-Based Assessment

The rule-based scorer achieved exact agreement with the final ground truth annotations across all 30 repositories following iterative refinement. Applied to the full 188-repository dataset, the scorer revealed substantial variability in structural FAIR compliance, with a mean score of 3.86 out of 6. README files and license files were near-universal (100% and 97% respectively), while dedicated software metadata files (31%) and persistent identifiers (51%) were considerably less common.

5.7.3 LLM-Based Assessment

LLM-human agreement was fair overall, with a macro-average weighted Kappa of 0.329 against the primary annotator and 0.275 against the second annotator. The LLM systematically assigned lower scores than human annotators, most notably for D5 (examples and tutorials) and D2 (usage clarity). These dimensions are particularly sensitive to the README-only evaluation design because relevant usage material is often provided through external documentation links that were not followed by the LLM. The strongest agreement was observed for D1 (installation clarity, κ=0.605), where instructions are typically provided inline. The LLM did not reproduce manual assessments with the same level of consistency observed between the two human annotators.

5.7.4 Hybrid FAIRness Profile

Spearman's rank correlation between rule-based and LLM totals was weak and not statistically significant ($\rho = 0.175$, $p = 0.356$). No clear monotonic association was therefore detected in the evaluated sample. This result supports treating the two components as complementary assessment dimensions rather than combining them into a single score.

5.7.5 Baseline Comparison

The howfairis comparison showed that the proposed framework extends the scope of a software-oriented metadata assessment by adding six documentation quality di-

mensions concerned with installation, usage, purpose, dependencies, examples, and contribution guidance. The F-UJI pilot provided a different finding: all five tested GitHub repository URLs received the same FAIR score of 42.31% despite substantial differences in their rule-based and documentation quality profiles. Together with F-UJI's warning that the evaluated resources were not identified as datasets, this indicates limited applicability of F-UJI as a direct quantitative baseline for the software repositories evaluated in this study.

5.7.6 Research Question Responses

RQ1 — Does LLM-based assessment improve FAIRness evaluation compared to rule-based methods alone? The LLM-based component extends the evaluation beyond structural repository indicators by assessing six qualitative documentation dimensions that cannot be captured reliably through simple file presence checks. The weak and non-significant association between the rule-based and LLM totals ($\rho = 0.175$, $p = 0.356$) indicates that no clear monotonic relationship was detected between the two components in the evaluated sample. The LLM component therefore provides additional, complementary information. However, its agreement with manual annotations was only fair overall, so it should be interpreted as an assistive assessment rather than a replacement for human judgement.

RQ2 — What are the reliability and limitations of LLM-based assessment for research software repositories? LLM-human agreement varied considerably across dimensions, from slight (κ=0.156 for D5 against the second annotator) to moderate (κ=0.605 for D1 against the primary annotator). The most significant limitation identified was the systematic underscoring of dimensions that depend on external documentation links, particularly D2 and D5. This pattern is partly associated with the README-only evaluation design and should be considered when interpreting LLM-based scores. Furthermore, the model's self-reported confidence was not consistently calibrated with agreement against human annotations, indicating that confidence labels cannot be treated as direct estimates of scoring reliability. No evidence of systematic README-length or verbosity bias was observed in the evaluated ground-truth set ($\rho = -0.169$, $p = 0.371$). This suggests that the systematic underscoring observed for D2 and D5 is better explained by the README-only evaluation design and its treatment of externally linked documentation than by README length itself.

RQ3 — Does the hybrid approach identify FAIRness gaps that existing tools miss?

The comparison with howfairis shows that the proposed framework assesses twelve indicators and dimensions, including six documentation quality dimensions not covered by howfairis. The framework therefore reveals documentation-related weaknesses, such as unclear installation, usage, examples, and contribution guidance, that are not represented by the metadata-oriented baseline. The weak association between rule-based and LLM totals further indicates that the two components provide different types of assessment information.

The F-UJI applicability experiment provides additional evidence from a data-oriented FAIR assessment perspective. F-UJI did not differentiate between the five tested GitHub repository URLs despite substantial variation in their software-specific assessment profiles. It was therefore not suitable as a discriminative quantitative baseline under the tested setup (Section 5.6.1). Together, the baseline results support the use of a software-oriented hybrid approach that combines structural repository indicators with qualitative documentation assessment.

6 Discussion

7 Conclusion

Bibliography

[1] Barker, M., et al.: Introducing the FAIR principles for research software. Scientific Data 9(622) (2022)

[2] Chue Hong, N., Breitmoser, E., et al.: D5.2 - metrics for automated FAIR software assessment in a disciplinary context. Tech. rep., FAIR-IMPACT Project (2025), v1.1

[3] Dvivedi, S.S., Vijay, V., Pujari, S.L.R., Lodh, S., Kumar, D.: A comparative analysis of large language models for code documentation generation. In: Proceedings of the 1st ACM International Conference on AIware (AIware 2024) (2024)

[4] FAIRsFAIR: F-UJI automated FAIR data assessment tool (2021), `https://www.f-uji.net`

[5] Hou, X., et al.: Large language models for software engineering: A systematic literature review. ACM Transactions on Software Engineering and Methodology (2024)

[6] Imani, A., Radmanesh, S., Ahmed, I., Moshirpour, M.: Does documentation matter? an empirical study of practitioners' perspective on open-source software adoption. arXiv preprint arXiv:2403.03819 (2024)

[7] Landis, J.R., Koch, G.G.: The measurement of observer agreement for categorical data. Biometrics 33(1), 159–174 (1977)

[8] Prana, G.A.A., Treude, C., Thung, F., Atapattu, T., Lo, D.: Categorizing the content of GitHub README files. vol. 24, pp. 1296–1327 (2019)

[9] Sharma, A., Sowe, S.K., Kim, S.Y., Hoseini, S., Limani, F., Boukhers, Z., Lange, C., Decker, S.: FAIR data assessment using LLMs: The fair-way. In: Proceedings of the 34th ACM International Conference on Information and Knowledge Management (CIKM '25) (2025)

[10] Treude, C., Middleton, J., Atapattu, T.: Beyond accuracy: Assessing software documentation quality. In: Proceedings of the 28th ACM Joint European Software Engineering Conference and Symposium on the Foundations of Software Engineering (ESEC/FSE '20). pp. 1509–1512 (2020)

[11] Wilkinson, M.D., Dumontier, M., Aalbersberg, I.J., et al.: The FAIR guiding principles for scientific data management and stewardship. Scientific Data 3, 160018 (2016)

[12] Zheng, L., Chiang, W.L., Sheng, Y., Zhuang, S., Wu, Z., Zhuang, Y., Lin, Z., Li, Z., Li, D., Xing, E.P., Zhang, H., Gonzalez, J.E., Stoica, I.: Judging LLM-as-a-judge with MT-bench and chatbot arena. In: Advances in Neural Information Processing Systems (NeurIPS) (2023)